

Proof of Personhood Without a Registry

A measured zero-knowledge construction for one-human-one-action,
and the four optimisation paths that failed

Rehan Malik · Independent Cryptography Researcher · github.com/rehanmalik2008

Version 1.2 · Apache-2.0 · © 2026 Rehan Malik · Reference implementation, circuits and raw measurements accompany this document; cite via `CITATION.cff`

v1.1: the single epoch counter is split into `epoch_action` (rate-limit window) and `epoch_tree` (tree cadence), closing one open problem from §7.2. Zero added constraints; one added public signal ($6 \rightarrow 7$). v1.2: an adversarial self-audit whose determination result is now solver-independent — a linear-time structural forward-determination walk shows every signal and both public outputs uniquely determined by the inputs, except seventy-two hint signals proved to influence no public output, with no Poseidon assumption and no linearization bound (§7.3–7.4); plus a simulated memory ceiling with a measured failure threshold (§7.5), and a trusted-setup ceremony coordinator built and tested but not run (§7.6). No gate closed.

Abstract

The internet cannot check that a request comes from a distinct human. Current fixes reintroduce a central authority: a biometric registry, a government identity provider, or a hardware vendor's attestation root. This construction removes the registry.

A device holds a secret s and publishes $C = \text{Poseidon}(s)$. Independent attesters sign C once, and a backend appends C to an append-only Merkle tree. Every later proof proves *set membership* rather than re-verifying signatures, so signatures are checked once per credential lifetime, off the hot path. The login circuit is 4,309 RICS constraints over BN254 and emits 7 public signals. Attester identity never appears in it, so it cannot leak from it. Witness upkeep is a public broadcast, identical for every client, so no per-user query occurs.

On a Samsung Galaxy A12 (Helio P35, 8×Cortex-A53, 3,751 MB RAM) the proving step takes 477 ms with a native prover and 3,374 ms in the browser, warm medians over 20 runs. Peak prover resident memory is 361 MB, 9.6% of device RAM. Four attempts to close that gap failed, each reported with the measurement that ended it. The result is a deployment split: native for entry-level hardware, browser above it.

Security rests on a priced assumption, $\gamma \geq \beta \cdot m \cdot E$, rather than a boolean. It has not been audited. Of six production gates one is closed; a new under-constraint check placed Gate 1 further from closure, not nearer. Section 6 states what it does not protect; Section 8, what would refute it.

1. The problem

A server receiving a request cannot tell whether a human or a program sent it. Every application that allocates something per person depends on being able to tell: one signup per person, one vote per person, one airdrop per person, one review per person. When the check fails, the cheapest attack is to be many people at once. Douceur named this the Sybil attack and proved the general result: without a trusted certifying authority or a resource cost per identity, a single entity can present arbitrarily many identities [3].

CAPTCHAs were the practical answer for about twenty years, and they have stopped working. Vision and language models now solve the standard challenge families faster and more accurately than people do,

and solving services buy human labour for a fraction of a cent per challenge when a model is not enough. Once a machine passes the test more reliably than a person, the test no longer measures what it was built to measure.

Three replacements are deployed or under serious proposal. All three work, and all three put back a central authority.

The biometric approach has a central operator enrol an iris or a face, store the template, and answer whether a given person has been seen before. A biometric template cannot be revoked and reissued the way a key can, and a person has only one iris, so a breach of the operator, a coercion order against it, or a decision to shut it down leaves the enrolled population with no recovery path. The operator also becomes the single party that decides who is counted as a person.

Government electronic identity, such as eIDAS 2.0 and the schemes modelled on it, gives strong assurance to anyone who holds documents. It makes participation depend on a state issuing those documents and continuing to issue them, which leaves out the undocumented and the stateless and gives a government a lever over who is allowed to act. It also inserts the state into transactions it previously could not see.

Hardware attestation through a secure element is cheap and works well, but it moves the root of trust to Apple, Google, or a chip vendor. Personhood then reduces to holding a device those firms have signed, and those firms gain a veto over who can act online.

The shared structure is that one party holds a record of who is real and verifiers query that party. The registry is the problem. The rest of this paper describes a construction with no registry of humans: there is a public set of cryptographic commitments, and no entry in it can be linked to a person, to an action, or to another entry.

1.1 What is claimed

A party proves to any verifier that it is a distinct human who has not made this claim before in this context, with:

- no central registry of people;
- no biometrics and no hardware root of trust;
- no revelation of which human;
- no linkability across contexts, by verifiers, by colluding verifiers, or by the issuer.

This is a personhood primitive. It answers "one human, once, here" and gives no answer to "which human". Not being able to identify anyone is the design goal, and it is also the boundary of what the primitive is good for.

2. Construction

The design separates a heavy operation that runs rarely from a light one that runs on every use. Verifying an attestation inside a circuit is expensive, so it is done once at enrolment and then carried forward by set membership.

Time is measured in two counters. `epoch_action` is the rate-limit window and the period over which one action per context is allowed; it is fast, for example ten minutes. `epoch_tree` is the cadence on which the membership tree publishes a new root; it is slow, for example daily. They are independent public inputs. Version 1.0 used one counter for both; splitting them lets a deployment rate-limit tightly without rebuilding the tree tightly, and costs zero circuit constraints.

2.1 Phase 1: enrolment, once per credential lifetime

1. The device generates a root secret s and computes the commitment $C = \text{Poseidon}(s)$. The secret never leaves the device.
2. k independent attesters sign C blind. Attester categories are social, institutional, temporal (a verifiable-delay-function anchor on presence), hardware (WebAuthn), and governmental eID. Hardware and eID are permitted as one input each. Neither is permitted as a root and neither is required.
3. The device proves the k signatures in a circuit of 13,099 constraints. The backend verifies that proof and appends C to a public append-only Poseidon Merkle tree of arity 8 and depth 9, capacity $8^9 = 134,217,728$ leaves.
4. The device caches its membership witness, which is 9 sibling groups.

2.2 Phase 2: proof of personhood, every use

The login circuit proves four statements, with every input private except the outputs:

- $C = \text{Poseidon}(s)$, knowledge of the secret behind a leaf;
- C is in the tree whose root was published for `epoch_tree`, opened in-circuit, so C itself is never revealed;
- $N = \text{Poseidon}(s, \text{ctx}, \text{epoch_action})$, a per-context nullifier that rotates with the rate-limit window;
- a rate-limiting-nullifier Shamir share $y = s + N \cdot \text{signalHash}$. Because N depends on `epoch_action` and not on `epoch_tree`, the RLN slot is $(\text{ctx}, \text{epoch_action})$ [6, 8].

The public output is exactly seven field elements: $[N, y, \text{root}, \text{ctx}, \text{epoch_action}, \text{epoch_tree}, \text{signalHash}]$. Five are inputs the verifier already chose or knows. Two are outputs. The verification key reports `nPublic` = 7. The verifier learns that one attested distinct human acted once in this context in this rate-limit window, and nothing else.

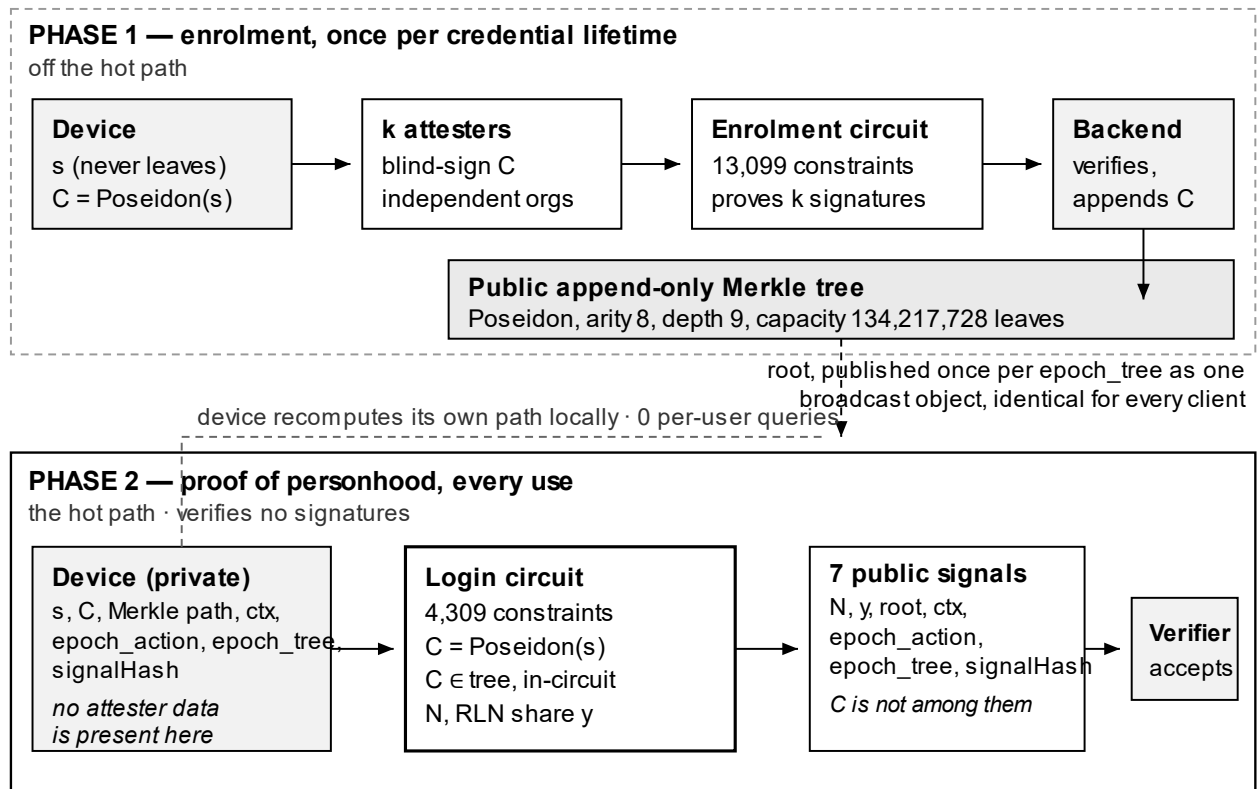


Figure 1. The two phases. Signature verification happens once, in Phase 1. Phase 2 verifies no signatures at all: it proves membership. No attester data enters the Phase-2 circuit, so none can leak from it. The nullifier binds to `epoch_action`; the tree root is the one published for `epoch_tree`.

2.3 Why membership instead of re-verifying signatures

This is the central design decision, and it was reached by measurement rather than by intent. The direct construction re-verifies k attester signatures inside the login circuit. Elliptic-curve signature verification is expensive in R1CS. One EdDSA-Poseidon verification over Baby JubJub costs about 4,207 constraints, and roughly 97% of that is two scalar multiplications, which R1CS cannot compress.

Several attempts to reduce that cost were built and measured. Halving the Merkle depth removed 7,290 constraints, and the circuit came to rest on the same signature floor. An RSA accumulator that took in-circuit membership out altogether did the same. In-circuit half-aggregation of the k signatures came out worse than verifying them directly, at 10,525 constraints against 8,890 for $k = 2$. Aggregation replaces k cheap fixed-base multiplications with one fixed-base and $k-1$ additional variable-base multiplications, and the variable-base multiplication is the costly gadget. Aggregation pays off outside a circuit because those multiplications can be batched into one; an R1CS has nothing to batch them into, so here the technique adds cost rather than removing it.

Moving signature verification out of the circuit worked, at 478 constraints, but the backend then learned which attestors had vouched. That destroys attester unlinkability, so it was rejected.

The resolution was to stop re-verifying signatures on the hot path. Attestation is a statement about a credential's origin. It only needs to be checked when the credential is created. After that, "this commitment was properly attested" is equivalent to "this commitment is in the set of properly attested commitments", which is a membership proof. Two consequences follow:

- **Cost.** Phase 2 fell from 13,099 to 4,309 constraints, a factor of 3.0. A wide, shallow Poseidon tree is what makes membership cheap: a Poseidon(W) hash costs roughly linearly in state size, but the tree needs only $\log_W(n)$ levels, so wider and shallower wins. Arity 8 at 134 million leaves costs 3,831 constraints for the opening; a binary tree at the same capacity costs 6,558, which is 71% more.
- **Privacy.** Attester unlinkability became structural rather than argued. The Phase-2 circuit contains no attester public key, no signature, and no category label. A property enforced by absence cannot be undone by a bug in a constraint.

2.4 Witness maintenance without queries

A membership proof needs a current Merkle path. The naive way to keep one current is for the client to ask the backend for its path. That request carries the client's leaf index, which is a stable identifier for that user, and it is issued on a schedule correlated with the user's activity. It would undo cross-context unlinkability outside the circuit, where the circuit cannot defend it.

The tree is append-only, so it does not need to be queried. Once per epoch_tree the backend publishes one broadcast object containing the appended frontier nodes. It is byte-identical for every client and contains no leaf index, position, or path. Each device recomputes its own affected path levels locally, 9 Poseidon(8) hashes, and asserts that the recomputed root equals the broadcast root, which also detects a divergent broadcast. No per-user request exists at any point. Because epoch_tree is decoupled from epoch_action, this republish cadence is set by tree economics, not by how tight the rate limit needs to be.

This is enforced structurally, not by policy. The server object exposes an epoch-keyed API with no method that takes a leaf index. The client refresh functions cannot reach the network and reject any broadcast carrying a position-like field. Five such guards are in place, and the test suite asserts zero upstream per-user calls across every scenario.

WITNESS REFRESH — one broadcast, every device, no queries

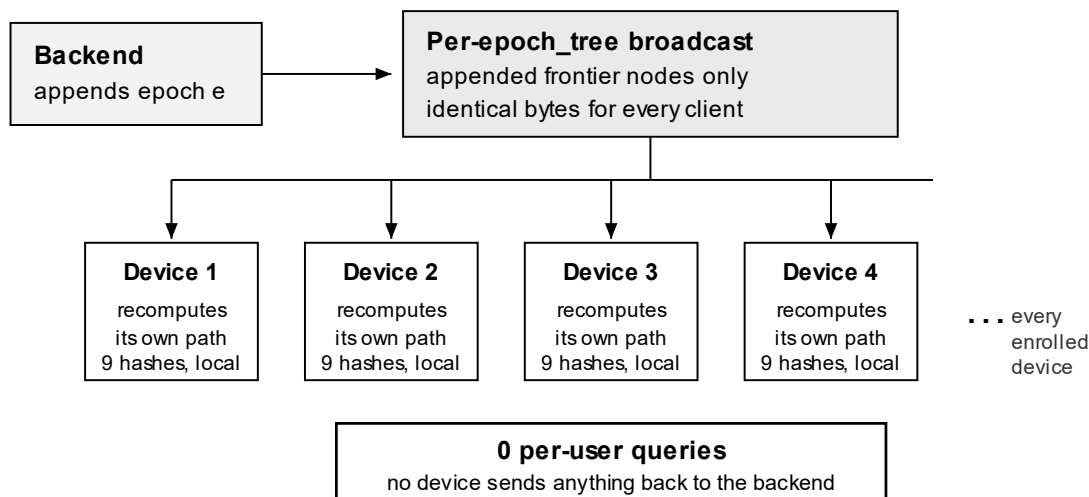


Figure 2. The per-epoch_tree broadcast is one object, byte-identical for every client, carrying no leaf index or path. Each device folds it into its own cached witness locally. Nothing flows from a device to the backend, so there is no request that could carry a stable per-user identifier.

2.5 Choosing k , and the diversity predicate

k is the number of independent attestations required at enrolment. It is a security parameter, not a constant. Setting it high raises the cost of forging a credential and also raises the friction for honest users, so it should be derived rather than assumed. With context-scoped attestation ($m = 1$) and attestation cost A per attester, the requirement of Section 4 gives $k = \lceil \beta \cdot E / A \rceil$, floored by whatever collusion tolerance the deployment wants. For low-value bot defence, $k = 2$ is often sufficient.

The diversity predicate D requires that the k attestations come from distinct categories, and in a production deployment from distinct organisations and jurisdictions. It costs 2 constraints per pair in the enrolment circuit, so it is close to free, and it is enforced in Phase 1 where the attester public keys are available. Phase 2 carries no category labels, which is why D cannot be checked there and does not need to be.

D has a structural consequence worth stating. A single organisation cannot satisfy a predicate that requires independent attesters, because its own attesters are not independent of each other. Any single-vendor deployment must either include external attesters, which means participating in a network it does not control, or drop D , which makes its variant measurably weaker. The security rule penalises centralisation directly rather than relying on governance to prevent it.

2.6 Relation to prior constructions

The components are not new. Anonymous credentials date to Chaum [1] and were made practical by Camenisch and Lysyanskaya [2]. Proving set membership in zero knowledge over a Merkle tree, with a nullifier to prevent double signalling, is the Semaphore construction [6]. The rate-limiting nullifier that burns a reused secret is Shamir secret sharing [7] applied as in [8]. Groth16 [4] and BN254 [10] are standard.

Two things here are specific. The first is moving signature verification entirely off the proving path, which is what makes the login circuit 4,309 constraints and makes attester unlinkability structural instead of argued. The second is that witness upkeep is a public broadcast with no per-user query, which closes a linkability channel that sits outside the circuit and that a circuit therefore cannot defend. Neither is a new primitive. Both are consequences of taking the end-to-end argument [5] seriously about where a property has to be enforced.

3. Security properties

Each property below is a consequence of the construction, not of an operational promise. The right-hand column states the strongest adversary the property survives.

Property	Mechanism	Strongest adversary survived
Sybil resistance	$\gamma \geq \beta \cdot m \cdot E$ (Section 4) plus the diversity predicate D	Economically rational attacker
Unforgeability	A fabricated C has no membership witness	Anyone not breaking the accumulator
Cross-context unlinkability	$N = \text{Poseidon}(s, \text{ctx}, \text{epoch_action})$; C never revealed	Colluding relying parties <i>and</i> the issuer
Attester unlinkability	No attester data exists in the Phase-2 circuit	The verifier itself
One action per context per epoch_action	Nullifier uniqueness; N binds to epoch_action, not epoch_tree	—
Double-use burn	RLN: two proofs in one (ctx, epoch_action) reveal s, for any epoch_tree	Self-enforcing; no adjudicator needed
No shared key	Membership replaces any MAC or token bridge	Removes the symmetric-key dilemma

Table 1. Security properties and the adversary each survives. Cross-context unlinkability holds against the issuer because the issuer sees only C at enrolment and only N at use, and C never appears in a Phase-2 proof.

The double-use property deserves a note because it is unusual. A rate-limiting nullifier is a degree-1 Shamir share: $y = s + N \cdot \text{signalHash}$. One proof reveals one point on a line and nothing about s . Two proofs in the same (ctx, epoch_action) with different signals reveal two points, and two points determine the line, which yields s . Losing the credential is the penalty for proving twice in one window, and it follows from the arithmetic rather than from a moderator's ruling.

The burn binds to epoch_action alone. N is $\text{Poseidon}(s, \text{ctx}, \text{epoch_action})$ with no dependence on epoch_tree, so two proofs in one rate-limit window carry the same N and lie on the same line even if they cite different tree roots. Varying epoch_tree is not a way out. If the nullifier had been bound to both counters, an attacker acting twice in one window against two different published roots would produce two distinct nullifiers and escape the burn; a test asserts the correct binding by recovering s from exactly that pair of proofs (scripts/test_rln_burn.mjs).

CROSS-CONTEXT UNLINKABILITY

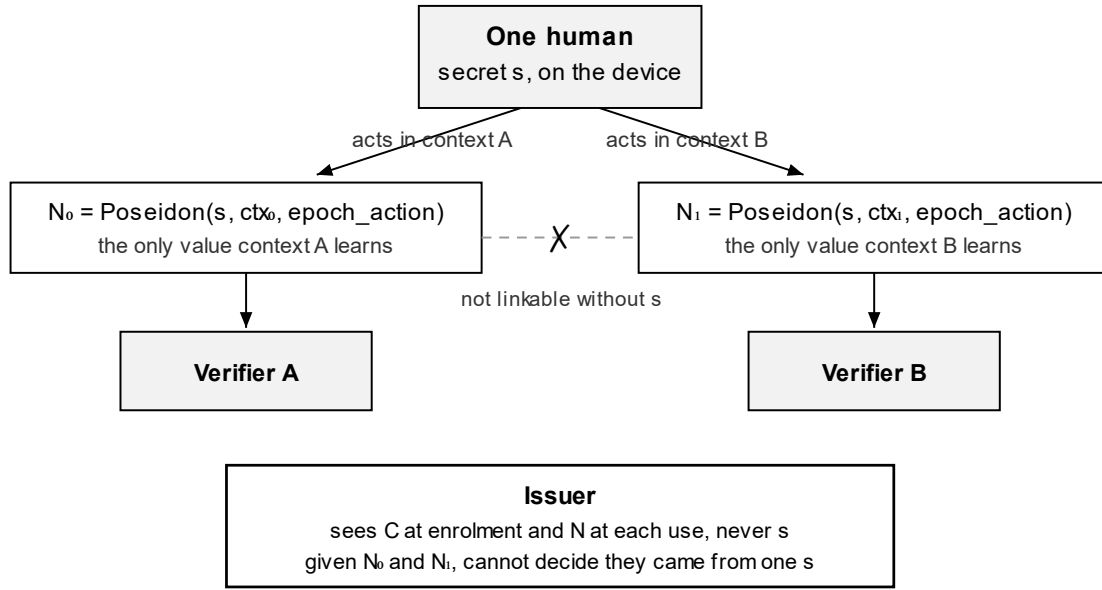


Figure 3. The privacy claim made concrete. The same s produces a different nullifier in every context. Each verifier sees only its own nullifier. Without s , no party, including the issuer that has seen both C and every N , can tell that two nullifiers belong to the same person.

4. Economics

Sybil resistance in this construction is an economic property rather than a cryptographic one. The cryptography guarantees that identities are countable and cannot be forged outright. What keeps an attacker from creating many of them is that each one costs money to mint. Getting that cost wrong is the most likely way for a deployment to fail, so the requirement is stated below as arithmetic.

4.1 The requirement

Let β be the value to an attacker of one fake identity in the application being attacked. Let γ be the cost of minting one credential. The naive condition is $\gamma \geq \beta$.

That condition is wrong, and the error is expensive. A credential is not consumed by one use. If it can act in m monetisable contexts, and it survives E epochs before expiry or burn, then one credential returns $\beta \cdot m \cdot E$ to the attacker. Deterrence therefore requires:

$$\gamma \geq \beta \cdot m \cdot E$$

A concrete case shows the size of the mistake. Suppose β is \$0.50, a credential reaches 20 monetisable contexts, and it lives 50 epochs. Then $\beta \cdot m \cdot E$ is \$500 while β is \$0.50. Pricing the mint at $\gamma = \beta$ underprices the defence by three orders of magnitude. The system would look secure and would not be.

4.2 Two ways to satisfy it, and why one is much better

The inequality can be met from either side. Raising γ to cover the widest reachable context set is simple. At $\beta \cdot m \cdot E = \500 it pushes the cost of being a person into the hundreds of dollars, which excludes the humans the system exists to serve. Result 1's own failure mode applies: if the price of being human is too high, you exclude humanity.

Shrinking m is better. Bind the attestation to a context class, so one credential cannot spray across unbounded contexts. Then $m = 1$ and the requirement becomes $\gamma \geq \beta \cdot E$. For bot defence, where β is measured in cents, the required γ lands in the range of about \$0.05 to \$2.00. That is cheap enough to be tolerable for honest users and expensive enough to make bulk forgery unprofitable. The cost is portability: a context-scoped credential is less convenient than one that works everywhere. That is a real trade and it should be stated to users rather than hidden.

4.3 Personhood is a dial, not a boolean

The consequence is that no single global answer to "is this a human?" exists. Each verifier faces a different β . A comment box and a financial account are not defending against the same attacker and should not buy the same amount of deterrence. Each verifier sets its own k , its own stake requirement, and therefore its own γ , to clear its own β .

Any system that returns `human: true` with no price attached is reporting a boolean where the underlying quantity is continuous. It is claiming more than it can support. The construction here exposes the dial rather than hiding it.

One caveat on the economics is that β has never been measured for a real application in this work. The ranges above are estimates. Section 8 lists this as a falsification criterion.

5. Measured results

Every number in this section was measured on the hardware named with it. Numbers derived by extrapolation are labelled "projected".

The reference device is a Samsung Galaxy A12 (model SM-A125F): MediaTek Helio P35, eight in-order Cortex-A53 cores, 3,751 MB RAM, Android 12, Chrome 152, retail price around \$110. It was picked as a deliberate low-end target. The people who most need Sybil resistance are disproportionately on hardware like this, so a result that only held on a flagship would not reach them.

5.1 Circuit size

Circuit	Constraints	Public signals	Role
wedge_direct_k3	13,099	3 in	Phase 1, enrolment, once per credential lifetime
wedge_mem_w8_d9_split	4,309	7	Phase 2, login, every use (split epoch)
wedge_mem_w8_d9 (pre-split, v1.0)	4,309	6	constraint-identical; the benchmark measurements in §5.3 were taken on this
+ revocation, R = 64	4,373	6/7	attester-level non-membership, +1.5%
+ revocation, R = 256	4,565	6/7	+5.9%
+ revocation, R = 1024	5,333	6/7	+23.8%

Table 2. Constraint counts from `snarkjs r1cs info` on the compiled R1CS. Groth16 over BN254. Tree capacity 134,217,728 leaves. The split-epoch circuit adds **zero constraints** and one public signal (`epoch` becomes `epoch_action` plus `epoch_tree`); the revocation costs are unchanged and apply on top of either. Revocation adds exactly **+R** constraints, all cheap linear checks.

A grep of the Phase-2 compile graph for signature gadgets returns zero matches. The verification key reports `nPublic = 7`. Both are mechanical checks a reviewer can repeat.

5.2 Memory

An early concern was that the browser prover requests a 2,048 MB WebAssembly linear memory at instantiation, which a 2 to 3 GB phone might refuse. Measuring resident memory rather than the request settled it.

Condition	Peak prover RSS	Share of 3,751 MB	Tab survived
Baseline, no background load	361.5 MB	9.6%	Yes, 20/20 runs
Under 900 MB ballast (3 tabs $\times$ 300 MB)	378.8 MB	10.1%	Yes, 20/20 runs

Table 3. Kernel VmHWM for the renderer process running the proof, captured over adb. No out-of-memory event and no lowmemorykiller activity in either condition.

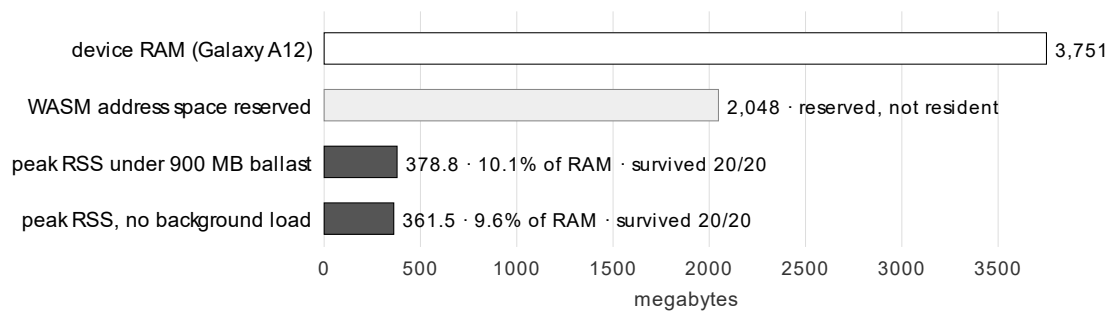


Figure 4. Memory, to scale. The reserved address space is what an early in-page measurement reported and it is 5.4 $\times$ the actual resident peak. The proof fits in about one tenth of the device.

The 2,048 MB figure is reserved address space, of which about 18% is committed. Reserved memory is not resident memory. An in-page measurement using `measureUserAgentSpecificMemory` reports roughly 21 MB, which is also wrong: it misses worker-thread WebAssembly entirely. Neither figure should be used. The kernel high-water mark is the number that decides whether a tab survives.

This does not close the memory question. The specified gate requires a device with 3 GB or less. The A12 has 3,751 MB. The result is strong evidence and is not closure. It is listed as an open gate in Section 7.

5.3 Proving time

All three engines proved the same circuit from the same proving key and the same witness. Every proof was verified against the same snarkjs verification key, and the public signals were compared byte for byte and found identical. The engines are interchangeable at the protocol level.

These on-device measurements were taken on the pre-split circuit (`wedge_mem_w8_d9`, v1.0). The split circuit is constraint-identical (4,309, Table 2) and differs only by one extra public input, so the prover times carry over unchanged; on desktop, Node, the two circuits prove within run-to-run noise of each other. The snarkjs and C++-to-WASM engines were re-run on the split circuit and verify identically with the new 7-signal public output (Appendix A); rapidsnark native was not re-run because it needs the A12, and cross-engine equivalence is engine-independent.

Engine	Cold	Warm median	Warm p95	Install
snarkjs, browser, deployed today	5,187 ms	3,374 ms	—	none
C++ compiled to WASM (rapidsnark via Emscripten)	3,071 ms	2,467 ms	2,671 ms	none
rapidsnark, native	510 ms	477 ms	626 ms	app or SDK

Table 4. Proving step only, Galaxy A12, 20 runs per engine. Witness calculation in the browser adds 1,164 ms warm and 1,592 ms cold on top of the browser figures.

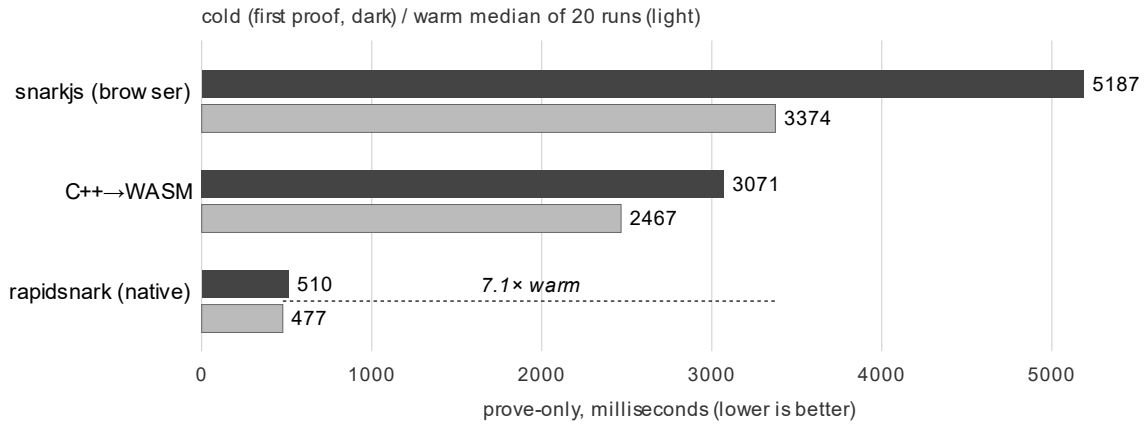


Figure 5. The same data. All three engines prove the same statement from the same proving key, and all three proofs verify against the same verification key with identical public signals (Appendix A).

The native prover runs 7.1 times faster than the browser prover on the warm median and 10.2 times faster from cold. For the native prover the first proof after startup costs about 7% more than a warm one; for the browser it costs 54% more. A real user proves once, from cold, so the browser number that matters in practice is the larger one.

5.4 Four optimisation paths that failed

The gap between 477 ms native and 3,374 ms in the browser is the difference between a zero-install product and one requiring an app on entry-level hardware. Four ways to close it were attempted. All four failed. Each is reported with the measurement that ended it, because the reasons are more useful than the outcome.

Path 1: free browser levers. Threading was already active. Cross-origin isolation was enabled and `crossOriginIsolated` was true on the device. Measured threading speedup was $2.51\times$ (12,580 ms single-thread to 5,011 ms with 8 threads, full prove, warm median). The browser supports WebAssembly SIMD, but snarkjs 0.7.6 ships no SIMD build and does not use `SharedArrayBuffer` for the prover. There was nothing unclaimed to switch on.

Path 2: compile the C++ prover to WebAssembly. rapidsnark was compiled with Emscripten, using pthreads, SIMD, and mini-gmp with 64-bit limbs in place of GMP. The build works and its proofs verify against the same verification key with identical public signals. It measured 2,467 ms warm, which is $5.17\times$ the native 477 ms. The projection for this path was $1.25\times$ to $2\times$ of native. The gate was set in advance at $2.5\times$, so the result failed by a factor of two. The build is $1.37\times$ faster than snarkjs on the proving step and $2.2\times$ faster cold, which is a real gain and is not enough.

Path 3: hand-written SIMD field arithmetic. Before writing any, the 5× was attributed. The prover's thread-pool size was made settable at runtime, and both engines were measured at matched thread counts. Native was constrained with taskset.

Threads	Native (ARM64 assembly)	WASM (portable C++)	Ratio
1	2,743 ms	13,319 ms	4.86×
2	1,442 ms	7,684 ms	5.33×
4	799 ms	4,229 ms	5.29×
8	518 ms	2,559 ms	4.94×

Table 5. Matched-thread comparison, Galaxy A12, same proving key and witness. Scaling from 1 to 8 threads is 5.30× for native and 5.20× for WASM. The single-thread WASM figure comes from a build with no pthreads and no memory growth, so it carries no threading overhead at all.

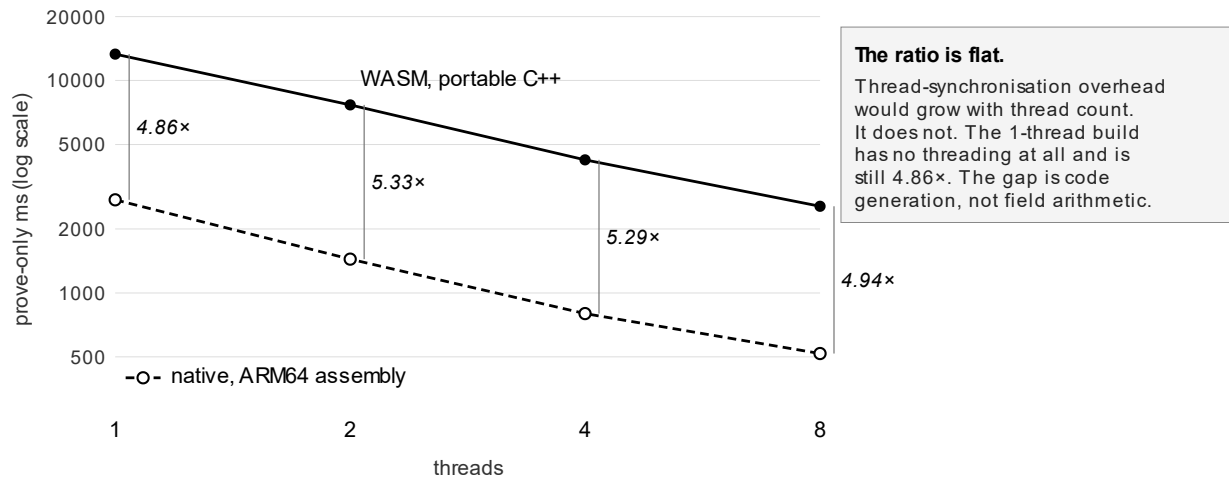


Figure 6. The measurement that ruled out writing field-arithmetic assembly. Both axes are the same circuit and the same proving key on the same handset. Log scale, so the two lines being parallel is exactly the claim: a constant factor, independent of thread count.

The ratio is flat. If the penalty were thread synchronisation, it would grow with thread count, because there would be more synchronisation to pay for. It does not grow. WebAssembly also parallelises as well as native here, 5.20× against 5.30× across the same range. A single-threaded build with every threading artefact removed is still 4.86× slower than native.

That result rules out field-arithmetic assembly. The gap is in code generation and runtime, and assembly for the field layer would target a layer that is not the bottleneck. The measurement cost one afternoon and avoided a multi-month effort aimed at the wrong place.

A second finding supports this. The best available WebAssembly field arithmetic is not SIMD. Reduced-radix scalar arithmetic with 29-bit limbs outperforms every simd128 approach in browsers, because WebAssembly lacks a 64×64→128 widening multiply and its SIMD is fixed at 128 bits. The achievable improvement over generic 64-bit CIOS is roughly 1.3× to 1.7×. The field layer therefore cannot close the gap on its own: its best available improvement is about 1.7× against a shortfall of five.

Path 4: a Rust prover with different code generation. arkworks compiled to WebAssembly was the remaining candidate. Inspection of ark-ff's Montgomery backend shows its only assembly path is gated on

x86_64 with BMI2 and ADX. For wasm32 it uses portable scalar Rust with unrolled CIOS, which is the same technique as the generic C++ path already measured, lowered through the same LLVM WebAssembly backend. Published browser figures for that stack on a Galaxy S23 Ultra, a flagship, give 793 ms for a comparable circuit. Scaled to the A12 that projects to roughly 1,900 to 2,100 ms, which is the same class as the 2,467 ms already measured. Changing the language the prover is written in does not change the LLVM WebAssembly back end that both languages compile through, and the constant factor lives in that back end.

The conclusion is that the $\sim 5\times$ belongs to the WebAssembly toolchain and browser runtime for this workload, not to this code. It should narrow as LLVM code generation improves and as relaxed SIMD and Memory64 mature. The engine is one C++ codebase, so that improvement arrives on recompilation.

5.5 Deployment matrix

Device class	Zero-install browser	Native SDK	Source
Entry-level (Galaxy A12 class, $\sim \$110$)	~ 4.7 s cold end-to-end via C++/WASM; ~ 6.8 s via snarkjs	~ 0.5 s proving step — GO	measured
Mid-range	$\sim 2\text{--}4$ s (<i>brackets the ≤ 2 s target; unmeasured</i>)	GO	projected
Desktop / flagship	~ 0.7 s full flow	GO	measured on desktop

Table 6. Deployment matrix. The circuit, proving key, nullifier construction, public signals, and privacy properties are identical in every cell. The split is a deployment choice, not a protocol fork. Desktop figure is an i7-9850H with Chrome, multi-threaded, full-flow p95 738 ms over 20 runs. The mid-range browser figure has not been measured on a mid-range device: the projected 2 to 4 s straddles the 2 s target, so on that projection some mid-range phones would clear it and some would not. Native end-to-end below ~ 1.1 s is projected; only the 477 ms proving step is measured.

The original target was 2 s for the full flow. That target assumed proving on every login. It has been revised, on the record, to 2 s on mid-range hardware and 5 s cold on entry-level hardware, proven once per epoch_action rather than once per login. Because the nullifier is scoped to epoch_action, a relying party can cache the result of a proof for that window: within one epoch_action the same human at the same relying party presents the same nullifier, so one proof covers it. The split does not force a fast epoch_action. A deployment that wants infrequent proving sets a long epoch_action and pays the several-second cost that rarely; a deployment that wants tight rate-limiting sets a short one and either re-proves that often or uses the native prover, where proving is 477 ms and the cadence does not matter. Before the split these two choices were the same knob. Deployments that must prove per login on entry-level hardware should use the native prover.

5.6 Witness maintenance at load

The tree publishes one broadcast per epoch_tree, a cadence set by tree economics alone. It is no longer tied to the rate-limit window: a deployment can rate-limit every ten minutes on epoch_action and still republish the tree once a day. The figures below are per epoch_tree, at various enrolment rates.

New enrolments per epoch_tree	Raw append log	Broadcast	Compression
1,000	31.3 KB	4.7 KB	6.6×
10,000	312.5 KB	44.9 KB	7.0×
100,000	3.1 MB	446.7 KB	7.0×

Table 7. Public broadcast size per epoch_tree, frontier-compressed. Identical bytes for every client.

Steady-state client refresh is 3 to 4 ms on desktop, projected at about 15 ms on a low-end phone, and is $O(\text{depth})$ regardless of enrolment rate. A client that goes dark for a month during a 100,000-per-epoch_tree surge downloads 13 MB and recomputes its witness in 122 ms on desktop. Upstream per-user calls during every test, including the month-offline catch-up: zero.

5.7 Verifier cost and integration surface

Verification is cheap and constant. Groth16 verification of the 7-signal proof measures about 20 ms in a browser on an i7-9850H, taken as the difference between full-flow and prove-only medians over 20 runs. Proof size is fixed at 3 group elements regardless of circuit size. The verifier does no elliptic-curve work proportional to the tree or to the number of users. On top of the zero-knowledge check it runs two constant-time freshness checks: the epoch_tree root within the last w published roots, and epoch_action within a separate, tighter recency bound. Both are deployment config; a root older than w is rejected regardless of epoch_action.

Whether a relying party can adopt this without rearchitecting was tested rather than asserted. A proof of concept runs a real OpenID Connect provider and two relying parties over real HTTP, issuing real Groth16 proofs from the wedge_mem_w8_d9_split artefacts. The nullifier is carried inside a W3C Verifiable Presentation in the ID token's vp claim, and is surfaced to the relying party as the OIDC sub. Ten assertions pass:

- first login at relying party A is allowed;
- a replay in the same epoch_action is rejected with HTTP 403;
- login at relying party B by the same human is allowed;
- sub differs between A and B, which is the pairwise, sector-scoped property;
- neither subject is a prefix, suffix, or rotation of the other;
- sub rotates when epoch_action changes;
- A allows the same human again in the new epoch_action;
- advancing epoch_tree alone (a tree republish) does not grant a new action: the same human is still rejected with 403;
- advancing epoch_tree alone does not rotate sub;
- advancing epoch_action again rotates sub and re-allows the action.

The provider generated every proof and still cannot link them, because the linkage exists only inside Poseidon($s, \cdot$) and it does not hold s . Integration is one verification call of about 80 lines, a poll of a /roots endpoint, and a (sub, epoch_action) deduplication check. No new credential format and no custom protocol. The one semantic change a relying party must absorb is that sub is scoped to

epoch_action, so it is an action token and not an account key. The last three assertions show the decoupling directly: republishing the tree neither resets the rate limit nor rotates the subject.

5.8 Revocation, both paths

RLN stops one human from acting twice in a window. It does nothing about a credential that should no longer be valid at all: a lost device, or a compromised attester whose whole issuance batch must be undone. Two mechanisms cover the two cases, and both were built and measured against the 4,309-constraint split baseline rather than estimated.

Individual revocation, in circuit. The split circuit gains an in-circuit non-membership check of the credential commitment C against a published revocation list of R field elements. For each entry it enforces $(C - r_i) \cdot \text{inv}_i = 1$, which has no witness when $C = r_i$, so a revoked holder cannot generate a proof at all. The constraint cost is exactly $+R$, measured: 4,373, 4,565, and 5,333 constraints for R of 64, 256, and 1,024, against a 4,309 baseline. Prove time on an i7-9850H rises by 21 ms, 20 ms, and 123 ms over a 538 ms base, median of 12 rounds. The public-input count becomes $7 + R$, so the list is carried explicitly by the verifier. A production deployment would instead commit the list as one accumulator root and prove non-membership against that, moving verifier input back to $O(1)$ at a few hundred constraints per tree level; the public-list form here is the tested one.

Bulk revocation, tree rebuild. At an epoch_tree boundary the operator rebuilds with revoked leaves set to zero. Indices are not compacted, so every surviving leaf keeps its position and only the internal nodes on the revoked paths change. That makes the rebuild broadcast incremental. Measured on the arity-8 depth-9 tree, across occupancies of 10k, 50k, and 200k leaves and revocation fractions of 1, 5, and 10 percent, the tombstone broadcast ranges from 9.3 KB to 1.18 MB, and grows with the number of revoked leaves times the depth, not with tree size. Full republication of the same trees is 0.35, 1.74, and 6.98 MB, so the incremental form is 6 to 39 times smaller. An unaffected client folds the broadcast into its cached witness in 2.4 to 5.1 ms median, well under the 122 ms month-offline catch-up baseline from §5.6, and never queries the tree. A revoked client's cached witness stops hashing to the published root, its refresh throws, and a fresh proof against the new root fails because its path no longer verifies.

Nine assertions cover both paths in `scripts/test_revocation.mjs`: an honest credential proves; the same credential placed on the revocation list cannot produce a witness; after a tombstone rebuild that revokes one leaf, the revoked client's refresh throws and it cannot prove against the new root, while an unaffected client refreshes query-free and proves. The liveness cost of the bulk path is disclosed in §6: a rebuild interrupts every client for one refresh window.

6. What is not protected

This section is complete. A reader who stops here has an accurate picture of the system's limits.

The nullifier is personal data. N is a pseudonymous identifier under GDPR. The verifier learns no name, no C , and no device identifier, but N is not anonymous. Within one $(\text{ctx}, \text{epoch_action})$ a relying party has a stable handle for that user and can build a per-context profile of that user's actions. That is the intended function, because Sybil resistance requires a stable per-context identifier. Cross-context linkage remains prevented.

The nullifier is scoped to epoch_action and is not an account key. It is a per-window action token. In a new epoch_action the same user at the same relying party presents a different N. A relying party using sub as an account primary key must change that assumption. Persistent pseudonymous accounts would need a second, epoch-independent pairwise identifier, costing about 255 constraints and no new cryptography. It is not built.

Trust is not zero. The honest claim is "no single trusted party, given a 1-of-N-honest publication medium". Root equivocation is always detectable after the fact by any auditor holding the root history. It is *prevented* only under 1-of-N honesty across the client's root sources. An earlier version of this work claimed "zero trusted parties". That claim is retracted.

The 1-of-N-honest clause is now an *enforced client-side check*, not a stated assumption. Before proving, and again before accepting, the client fetches the hash-chained transparency head ($\text{head}_e = \text{Poseidon}(\text{head}_{\{e-1\}}, \text{root}_e, e)$) from N independent root sources and cross-checks. It classifies each reachable source as agreeing, lagging-but-on-the-chain (tolerated within the staleness window w), or disagreeing, and fails closed on any disagreement or below a quorum of reachable sources (default 2 of 3). This raises the adversary's requirement from "control the client's one root source" to "control every source the client queries, simultaneously and consistently." It does not defeat a full eclipse; see §7.2.

Independence of the root sources is normative. The check is worthless if the sources share a failure domain. They must differ in operator, in network path, and where possible in trust root: for example a public chain, a CDN-backed static endpoint, and a third-party bulletin board. Three endpoints on one operator's infrastructure are one source with three URLs, and would make the trust claim look stronger while changing nothing.

Timing correlation. Unlinkability holds per proof. It does not hold against traffic analysis. Witness refresh must run on a schedule independent of proving, and download sizes must be padded. These are normative requirements, not suggestions. A deployment that refreshes witnesses when a user logs in has reintroduced the correlation the broadcast design removed.

Attester collusion. k colluding independent attesters can mint fake credentials. The diversity predicate D requires attestations from organisations in different jurisdictions, which raises the coordination cost. It does not eliminate the attack. This is a sociological problem and it is not fully solvable by cryptography.

Bulk-revocation liveness. Revoking a compromised attester means rebuilding the tree without its leaves (§5.8). The rebuild is published as one broadcast and clients recompute locally, as for a normal epoch. But between the rebuild being published and a given client's next refresh, that client cannot prove: its cached witness no longer hashes to the current root. An attester compromise therefore causes a refresh-window interruption for the whole population. This is a liveness cost, not a security one, and it is the price of bulk revocation without a per-user query.

Credential rental and coercion. Anonymous credentials resist rental economically: oversubscription is unobservable to the buyer, contracts are unenforceable, and the seller can rotate freely. They are not impossible to rent. For bot defence this is acceptable, because a rented credential still consumes one real human's action budget, so renting redistributes a scarce human action rather than manufacturing bots. For defence against coordinated inauthentic behaviour it is weaker: an attacker renting from P humans obtains P coordinated genuine actions and bypasses γ entirely. Security in that setting additionally requires the

rental market to be illiquid. Human-presence gating is the sound application. Coordinated-behaviour defence carries this caveat.

State-scale adversaries. Every economic bound in Section 4 assumes an attacker constrained by solvency. An adversary who does not care about cost is not covered.

Not post-quantum. The construction uses BN254 and Groth16. A crypto-agility path exists, and migration follows the ecosystem rather than leading it.

Trusted setup. Groth16 requires a per-circuit trusted setup. A compromised setup breaks soundness, meaning forged proofs become possible. It does not break privacy. The proving keys distributed with this work come from a throwaway single-contribution setup and are for evaluation only. Production requires a public multi-party ceremony or migration to a transparent-setup proof system.

7. Open problems

7.1 Gates that block production

Five of the six remain open; all are fundable external work rather than open research, and none is a cryptographic unknown. Gate 6 is closed. Gate 1 gained a tool-independent under-constraint check and, with it, a clearer picture of its own distance from closure (§7.4): the self-audit tools had not been shown to catch a real defect in this circuit, and validating that surfaced one genuine under-constraint in a mutant and left Picus unvalidated as an oracle here. That check was then strengthened to the point where a linear-time structural argument — no solver, no assumption — shows every signal, and both public outputs, uniquely determined by the inputs except seventy-two hint signals proved to influence no public output (§7.3). That makes an eventual external audit cheaper and better targeted but does not substitute for it, because unique determination is one property an audit checks, not all of them. Gates 3 and 5 gained measured evidence without moving: a simulated memory ceiling with a measured failure threshold (§7.5), and a ceremony coordinator built and tested ahead of need (§7.6). None of that closes a gate, and none of it is presented as closing one.

#	Gate	Why it blocks	Status	Path to close, and who does that work
1	Two or more independent circuit audits	An under-constrained signal silently breaks soundness. Reproducible builds prove the binary matches the source; they say nothing about whether the constraint system is correct.	Open, and after v1.2 understood as further from closure than it looked. Mutation testing (§7.4) showed the self-audit's tools had never been shown to catch a real defect here; a two-witness search added to check that found one genuine under-constraint among the ten mutants, which the adversarial harness misses and Picus returns no verdict on. On the pristine circuit that search now runs at 56 structurally diverse witnesses with no direction moving a public output, the seventy-two benign hint freedoms are proved benign rather than assumed, and a linear-time structural forward-determination walk — no solver, no assumption, no linearization — establishes that every other signal, and both public outputs, are uniquely determined by the inputs (§7.3). All of that makes an external audit cheaper and better targeted; none of it is independent review, and — even with the constraint system proven perfectly determined — that is one failure mode an audit examines, not the whole of it. None closes this gate.	Two independent external circuit audits. Scoped and fundable; the mutation table and the compositional decomposition tell an auditor which bug classes the existing process already covers and where to look first.
2	GDPR right-to-erasure legal memo	An append-only tree versus a deletion right. Blocks EU deployment until analysed.	Open	One legal engagement with a data-protection counsel. Scoped, fundable, no research content.
3	Peak RSS and p95 on a device with 3 GB or less	The formal memory gate. The A12 at 3,751 MB is evidence, not closure.	Open. Simulated-ceiling evidence added (§7.5): the prover completes under a hard 336 MB cap and is OOM-killed only at 320 MB. Simulated on desktop x86; a memory ceiling is not a device. An Android emulator at 2–3 GB (the real OS, so the real memory manager) was attempted and not completed — the build host could not reach the Android SDK endpoint; harness and AVD setup are scripted and ready.	Acquire one entry-level handset with 3 GB or less and re-run the existing on-device harness, or run the scripted emulator path on a host with SDK access. Cheap, fast, no new code.
4	IP indemnification and liability structure	Enterprise contracts require it.	Open	Standard commercial and legal structuring. Follows a design partner rather than preceding one.
5	Trusted-setup ceremony, or migration to a transparent-setup system	Soundness assumption. Current keys are evaluation-only.	Open, disclosed. Coordinator built and tested against deliberately invalid contributions (§7.6); circuit set frozen for v1.2 (CIRCUIT-FREEZE.md); run-book written (docs/CEREMONY-RUNBOOK.md). The ceremony itself has not been run.	Run the ceremony against the frozen circuits with a pre-announced beacon and contributors genuinely independent of each other and the project. Infrastructure exists and is frozen against a target; the remaining step is recruiting independent contributors.

6	Attester-level revocation mechanism	RLN covers double-use only, not a compromised attester.	Closed. Both paths implemented, measured, tested (§5.8): an in-circuit non-membership check for individual revocation (+R constraints, measured +64 / +256 / +1,024), and an incremental tree rebuild for bulk revocation. Liveness cost disclosed in §6.	—
---	-------------------------------------	---	--	---

Table 8. Production gates. Gate 6 moved to Closed in v1.1; gates 1, 3 and 5 gained measured evidence in v1.2 without moving. The final column is a description of work already specified elsewhere in this paper, not a commitment to a date.

7.2 Research questions that remain

Resolved since v1.0, and so removed from the list below. First, *separating the rate-limit window from the tree publication cadence*. One counter used to serve as the nullifier epoch, the rate-limit window, and the tree cadence at once. It is now two independent public inputs, `epoch_action` and `epoch_tree`, at a cost of zero constraints. The RLN burn binds to `epoch_action` alone; a test recovers `s` from two proofs in one action window that cite different tree roots, confirming the binding (`scripts/test_rln_burn.mjs`). Second, *the bulk attester revocation trade*. It has now been measured rather than estimated (§5.8): the tombstone rebuild broadcast is incremental, 9 KB to 1.2 MB at realistic occupancy against 0.35 to 7 MB for full republication, and an unaffected client recomputes its witness in 2 to 5 ms. The periodic rebuild is the practical path; carrying every revoked commitment in `R` is not. See §7.1 Gate 6.

- **A measured β .** The economic parameter has never been measured against a live application. Every figure in Section 4 is an estimate. This is the single most valuable missing number.
- **Full eclipse.** A targeted eclipse is detectable given one honest root source, and the client now enforces that condition: before proving and before accepting, it cross-checks the hash-chained transparency head across `N` independent root sources and fails closed on any disagreement or below a quorum of reachable sources (§6). This raises the bar from controlling the client's one root source to controlling every source it queries, at once and consistently. It does not defeat a full eclipse. An adversary who holds all `N` sources simultaneously still passes the check, and that case is only auditable after the fact.
- **Diversity in practice.** Whether 3 to 5 genuinely independent attesters exist in any given jurisdiction is an empirical question about institutions. It has not been surveyed.
- **The WebAssembly gap.** The $\sim 5\times$ penalty is a toolchain property. Whether relaxed SIMD, Memory64, and improved code generation close it to the point where entry-level hardware needs no native prover is a question for the WebAssembly ecosystem rather than for this construction.

7.3 Self-audit of the constraint system

Gate 1 exists for one failure mode: an under-constrained signal, where the constraints do not fully pin a value, so a malicious prover can choose one the honest prover never would and still produce a verifying proof. The test suite cannot find this. Every test drives the honest prover. A self-audit was run against that blind spot. It does not close Gate 1, which requires two or more independent external reviews. The full transcript, with tool versions, commands, raw output, and reasoning for every dismissed finding, is `docs/SELF-AUDIT.md`.

How claims in this section are made. Every result below is reported only where a direct test was actually run, not where the structure made an outcome look certain. That rule was learned the hard way:

across the self-audit, the results that turned out wrong were wrong in one recurring way — they trusted an inference instead of executing the check — and each was caught later by running the check. The ledger, kept deliberately visible in docs/SELF-AUDIT.md: a tool crash was counted as a detection and inflated a 10/10 (corrected by checking what “detection” meant); an under-constraint was first called forgeable by the right reasoning about the wrong mechanism, then retracted on two hand-forgeries that never perturbed the one intermediate signal that mattered, then reinstated by a search that did (three swings, one signal); “9 of 10 caught” counted signals produced rather than exploitable bugs; and, earlier in the paper's history, a deterrence inequality was asserted in the wrong shape and corrected only by doing the arithmetic. Every one of those felt certain when written. The consequence adopted here is that inference from structure is not trusted without the direct test, even when it feels certain, because the test is cheap and the wrong published inference is not.

Static analysis. Two analyzers were run over every deployed circuit: the split login circuit, the three split revocation variants, the pre-split login circuit, and the enrolment circuit. Picus 138b151 (Veridise) decides one specific question: whether the constraints uniquely determine every public output from the inputs. It returns *the circuit is properly constrained* for all six. That verdict is now qualified. Picus has been shown to flag a blatant two-signal under-constraint, but on a small Poseidon-plus-RLN circuit with a deliberately freed output it returns *cannot determine*, and on the full circuit with the same defect injected it does not terminate within fifteen minutes (§7.4). A *properly constrained* verdict from a procedure that returns *unknown* or does not terminate on known-bad circuits of this structure is not a validated oracle. It carries weight here only because a second, independent method agrees: a tool-independent two-witness search (§7.4) finds no second satisfying witness that moves a public output on the pristine circuit, within its stated bound. circomspect 0.9.0 (Trail of Bits) reported nothing at any level on the instantiated entryptoints. On the parameter-symbolic templates it raised two warning classes, both on code that ships. The first: the public input `epoch_tree` has no in-circuit constraint. That is by design; it is prover-supplied and verifier-checked for root freshness, the same shape as `ctx` and `signal_hash`, and it is listed among the verifier obligations below. The second: the non-membership gadget assigns its inverse witness with the hint operator before constraining it on the next line. That is the standard safe pattern, identical in shape to circomlib's own `IsZero`; the assigned signal is uniquely pinned by `diff · inv === 1`, which is unsatisfiable exactly when the credential is on the revocation list. A solver verdict bounded by a query timeout is not a proof-assistant artifact, and neither tool replaces expert review.

Manual footgun pass. A per-template check for the known Circom hazards found, on the deployment path: no unconstrained output signal, one hint assignment (the safe one above), no missing range check reachable by a malicious prover, no component left with an unwired input, and no signal aliasing.

Adversarial witness harness. `scripts/test_soundness_adversarial.mjs` constructs 24 witnesses an honest generator would never produce and checks the circuit rejects each. It works two ways: hand a malformed input to the witness generator and require it to fail on a constraint, and take an honest witness, flip one signal to an adversarial value, and require both the RICS satisfaction check and a fresh proof to fail. A tampered assignment that still satisfied the constraints, or still verified, would be a real soundness bug. None did. The harness covers, each against a malicious prover rather than the honest path: a secret that does not match the opened leaf; a Merkle path of forged siblings; an out-of-range or mis-pointed path index; a proof against a root the path does not open to; a revoked credential attempting a non-membership

proof; a nullifier rebound to a different secret or forced to collide across contexts; an RLN share moved off its line or rewritten for a different secret; the enrolment commitment swapped; and the enrolment diversity and signature checks. It also recovers the secret s end to end from the two proofs a double-actor would produce in one $(ctx, epoch_action)$ window, confirming the burn is arithmetic, not policy.

Two-witness search. A tool-independent under-constraint oracle, added after the mutation round showed the other methods had never been shown to catch a real defect here. It works from the definition: a circuit is under-constrained at a signal when two distinct witnesses satisfy every constraint, agree on every input, and differ at that signal. The search (`scripts/two_witness_search.mjs`) takes the Jacobian of the constraint system at the honest witness, computes the null space of its restriction to the non-input signals over the scalar field, and re-checks each null-space direction against the full non-linear constraints. On a mutant built to be under-constrained at the RLN share it finds the exploit witness directly (§7.4). The bound is stated with the result: a second witness reachable only by a large non-linear jump with zero linear component is outside the search, and finding none is weaker evidence than finding one.

Multi-point probing. The Jacobian null space is a property of one point on the solution manifold, and a non-linear system's rank can vary across it, so the search was re-run at 56 structurally diverse honest witnesses of the deployed circuit (`scripts/two_witness_multipoint.mjs`): leftmost and rightmost leaf, maximum-depth and single-level and random path shapes; all-zero, all-maximal, random and alternating sibling vectors; and s , ctx , both epoch counters and $signal_hash$ at random values and at field edges. The null-space dimension is a stable nine at every one of the 56 points, no direction has any component on any public output at any point, and every direction is an `IsZero` inverse hint — the well-known benign internal freedom of that gadget, which simply relocates from `eq[i][0]` to `eq[i][path_index[i]]` as the path shape changes, with no change in the amount of freedom. Rank over the scalar field is exact, so there is no near-degenerate singular value to report: each of the nine free columns is an exact zero column of the Jacobian, and every other column is a pivot at every point. The quantified claim that replaces “no under-constraint found” is: *no second witness with a non-zero linear component was found at 56 structurally diverse honest witnesses, across the full R1CS with exact non-linear re-verification*, with the large-non-linear-jump bound unchanged.

Taint proof for the nine hint freedoms. That the nine `IsZero` freedoms are benign was, until now, asserted from folklore. It is now shown mechanically (`scripts/taint_iszero.mjs`): for each hint, its null-space support is exactly itself; its column in the linearised R1CS is an exact zero column, so nothing in the system responds to a change in it; perturbing the honest witness along that direction by six different field multipliers leaves all 4,309 non-linear constraints satisfied and N and y bit-identical; and the transitive closure of the directed constraint-influence relation from each hint contains no public output. The nullifier and RLN sub-circuit and the Merkle sub-circuit share only the commitment C , flowing into the tree; nothing flows back.

The circuit is heavily over-determined, and that is a positive result. Removing one binding constraint from it frequently does not create an exploitable weakness because the signals are redundantly determined: the commitment by the Merkle path against the public root, the path-index range check by the slot-selection constraint, the `IsZero` hints by nothing. Of the ten mutations tried, only the one that frees the RLN share produced a second witness that moves a public output. Redundancy is a real property of the

design worth stating; it also means a detection-rate metric over an unfiltered mutant set mixes real misses with correct non-detections of mutants that changed nothing observable.

Compositional verification. Picus does not terminate on the full circuit because its deduction does not propagate uniqueness through the Poseidon permutation. The standard response is to verify at the module boundary (`scripts/compositional_verify.mjs`). Layer A: the circomlib Poseidon gadget in isolation, at arities 3 and 8, is uniquely determining — the two-witness search returns null-space dimension zero at sixteen random input points for each, and Picus confirms *properly constrained* for arity 3 (it does not terminate on arity 8). Layer B: every Poseidon instance in the deployed circuit is replaced by a relation that asserts only determinism — equal inputs give an equal output, nothing about hashing — and the two-witness search on the resulting 217-constraint system returns null-space dimension nine, all of it the same benign `IsZero` hints, with N and y uniquely determined. Composing the two: given that circomlib Poseidon uniquely determines its output, the deployed circuit's public outputs are uniquely determined by its inputs. This does not make Picus usable here — it does not terminate on the abstraction either — and it is not a substitute for an audit. What it buys is a smaller circuit-specific check, 217 constraints instead of 4,309, resting on one assumption stated plainly: *circomlib's Poseidon RICS uniquely determines its output signals from its input signals*. That assumption is narrow, is shared by essentially every circom project that hashes, is supported by published analysis of the gadget, and is checked directly in Layer A for the two arities this circuit uses.

Removing the linearization bound. Every result above is a Jacobian argument, so each carries the same caveat: a second witness reachable only by a large non-linear jump, with zero linear component, is outside it. Fifty-six probe points repeat a local check; they do not make it global. The two-witness question can instead be posed exactly, as a satisfiability query over the scalar field — do two witnesses exist that satisfy every constraint, agree on every input, and differ at a public output — and handed to an SMT solver with a native finite-field theory (`scripts/ff_smt_two_witness.mjs`; `cvc5` 1.3.4 built with `CoCoALib`). An *unsat* answer is then a complete proof with no linearization and no bound. Three results follow. First, on the sub-circuit that actually computes the public outputs — $N = \text{Poseidon}(s, \text{ctx}, \text{epoch_action})$ and $y = s + N \cdot \text{signal_hash}$, with Poseidon abstracted — with *every input left symbolic*, `cvc5` returns *unsat* in 0.13 s for both outputs: an unconditional, universal, non-linear proof that N and y are uniquely determined by their inputs. The Merkle half feeds nothing back into them, so this is the load-bearing statement, and it now holds without a probe count. Second, on the full 217-constraint abstraction with inputs pinned to a concrete honest assignment, `cvc5` returns *unsat* in about a second at six structurally diverse points — an exact non-linear check, still one point at a time. Third, the fully symbolic query over the whole 217-constraint abstraction does not terminate, and neither does any query involving an unabstracted Poseidon permutation of realistic size: `cvc5`'s finite-field engine hits the same S-box ideal that stops Picus. So the finite-field solver removes the linearization bound for the public outputs and makes it point-wise exact for the whole abstraction, but the monolithic universal query does not terminate — and the next paragraph is why that no longer matters.

The structural argument: unique determination without a solver. Two architecturally unrelated tools — Picus deduces over the constraint ideal, `cvc5` computes a Gröbner basis — fail identically on any circuit with a real Poseidon, at a few hundred constraints. That is the signature of a category error: both ask a *semantic* question over the field algebra and pay the cost of the x^5 S-box ideal, but unique determination of

a forward-computable circuit is a *structural* property of the constraint graph. A constraint $(A \cdot w) = (B \cdot w) = (C \cdot w)$ in which every signal of A and B is already determined, and exactly one signal of C is not, with an invertible coefficient, determines that signal uniquely by division. Seeding the inputs and iterating to a fixpoint (scripts/forward_determination.mjs) is linear-time, calls no solver, and reasons about the field not at all. On an --01 build of the deployed circuit (11,921 constraints; --02 fuses constraints and is not in this form) it determines every signal except the seventy-two IsZero inverse hints — the <-- assignments no constraint pins, which is the correct structural picture of that gadget — in 0.5 s. Both public outputs are determined. The nine of those seventy-two that are free at the deployed witness are *exactly* the nine null-space directions the Jacobian search found, which is the cross-check that the two methods agree. The Poseidon permutation is covered with no assumption at all, because its R1CS is triangular by construction; the comparator outputs that forward substitution cannot reach — IsZero pins them by a case-split on whether its input is zero — are closed with the proven property of that verified gadget, a cited lemma, not a substitution step, and they do not touch the public outputs regardless. The determination claim for N and y is therefore now *unconditional and solver-independent*: no Poseidon assumption, no linearization bound, no probe count. The Poseidon-determinism assumption also has a one-line mathematical corroboration, recorded as a cited design property rather than a verification result: $x \mapsto x^5$ is a bijection on the field because $\gcd(5, p-1) = 1$ for BN254 (the reason $\alpha = 5$ was chosen), and the MDS layers are invertible, so the permutation is injective.

What this does not do. The circuit is proven *uniquely determined* — every signal is pinned by the constraints — which is one soundness property. An external audit additionally checks what determination does not touch: that the circuit computes the *intended* function — that the nullifier construction is the right one, that the RLN binding achieves what Section 3 requires of the burn, and that there is no logic error in *what* is being constrained. Determination establishes none of that. It makes an audit faster and better-targeted — a reviewer can skip re-deriving that the constraints pin the signals and spend the time on semantics — but it does not predict the audit's outcome, and Gate 1 stays open until two independent reviews are done.

Trust-assumption red-team. The residual trust is one honest root source among N. If the N mirrors share a certificate authority, a hosting provider, or a DNS resolver, a single compromise serves a consistent forgery to all of them and the client's cross-check passes. This is the full-eclipse case already listed above; the red-team's contribution is to name the concrete shared dependency by which N sources collapse to one. Independence in operator, network path, and trust root is stated as normative in Section 6; it is not enforced by code.

What the self-audit does not cover. Independent expert review. A machine-checked proof in a proof assistant. A validated solver oracle: Picus returns *properly constrained* on the deployed circuit but could not be shown to detect a known under-constraint in a circuit of this structure, so it is kept only as a corroborating third method. The determination result itself is now solver-independent and unconditional — the structural forward walk needs no Poseidon assumption and no linearization — but it is still *only* determination: it says every signal is pinned, not that the circuit is correct. The circomlib primitives are trusted as-is for their *algebraic* properties (collision and pre-image resistance); their R1CS determination is what the structural walk establishes directly. The verifier and deployment obligations the circuit deliberately does not enforce: epoch_tree freshness, the signal_hash binding, the authenticity of the published

revocation list, the backend's authoritative diversity check at enrolment, and the refresh-to-login timing separation. And the trusted setup, which is evaluation-only regardless of how well the circuit is constrained. Gate 1 stays open.

7.4 Mutation testing: what the self-audit would and would not catch

Section 7.3 reports that three methods found nothing. That is consistent with two different worlds: a sound circuit, or a circuit whose bug all three methods miss. Nothing in Section 7.3 separates them, because none of those tools had ever been shown to catch a bug in these circuits. Mutation testing separates them. A known bug is injected into a copy of a deployed circuit, and each method is asked whether it notices. The result is not a claim that the circuit is sound. It is a map of which bug classes this process detects.

Ten mutants were injected one at a time across the login, revocation and enrolment circuits, covering the classes that matter here: a constraint operator downgraded to a bare assignment, a deleted range check, an equality compared against the wrong term, the nullifier hash stripped of `ctx` or of `epoch_action`, the two epoch counters aliased to each other, the RLN share detached from `signal_hash`, the in-circuit non-membership constraint deleted, and the signature verifier disabled at enrolment. Nine of the ten produced a signal in at least one method, which is not the same as nine exploitable bugs; two of them turned out to have no demonstrated exploit behind them. *circumspect* caught seven, the adversarial harness six, *Picus* none. Each mutation is applied by an exact-string replacement that fails unless its anchor matches exactly once, so a mutation that did not apply can never be scored as undetected; that guard caught two anchors which would have mutated a superseded template rather than the deployed one.

The tenth mutant is not a detection gap. Deleting the `path_index` range check should let an out-of-range index through, and it does not: feeding the mutant an index of 8 or 99 produces exactly the same rejection as the pristine circuit, because the surviving constraint that pins the selected slot to the running hash already excludes it. With no slot selected the dot product is zero, which would require a Poseidon output to equal zero. The mutant accepts and rejects exactly the same witnesses as the original, which makes it an equivalent mutant rather than a survivor. The constraint costs nothing after optimisation and stays, because stating the intent explicitly is better than resting on a probabilistic argument. The automated runner did report it as a survivor; what corrected that was a behavioural test, not a tool.

Picus detecting none of the ten is mostly correct behaviour, and worth stating plainly because Section 7.3 could be read too generously. *Picus* decides whether the outputs are uniquely determined by the inputs. Nine of these mutants leave the circuit perfectly determined and merely wrong: a nullifier that drops `ctx` is still a function of its inputs. So "all six deployed circuits properly constrained" is a real result about under-constraint, and it says nothing about whether the circuit computes the intended function. It also could not be validated as an oracle here. On a blatant two-signal under-constraint *Picus* reports *underconstrained* and prints both witnesses; on a small Poseidon-plus-RLN circuit with an output freed it reports *cannot determine*; and on the full circuit with the nullifier or the RLN share freed it does not terminate within fifteen minutes, across every run tried. It answers quickly with *properly constrained* on the sound circuit and gives no usable answer on known-bad circuits of the same structure. A verdict from a procedure with that profile is worth weight only alongside an independent method that agrees, which the two-witness search does on the pristine circuit.

One mutant is genuinely under-constrained, and it took a tool-independent oracle and two wrong intermediate conclusions to establish that. The mutant that downgrades the nullifier assignment $N \leftarrow hN.out$ to $N \leftarrow hN.out$ is the canonical Circom soundness bug. circomspect flags it syntactically; the adversarial harness passes all 24 checks on it. A first analysis inferred from the compiled circuit that the nullifier was forgeable. A second analysis attempted two forgeries by hand, moving the nullifier alone and moving it together with the RLN share, saw both rejected, and withdrew the claim. Both intermediate analyses were incomplete: the second one never perturbed the internal product $a1x = N \cdot signal_hash$, so it could not have found the second witness even though one exists.

The two-witness search settles it. A circuit is under-constrained at a signal if two distinct witnesses satisfy every constraint, agree on every input, and differ at that signal. The search (`scripts/two_witness_search.mjs`) computes the Jacobian of the constraint system at the honest witness, finds the null space of its restriction to the non-input signals over the scalar field, and re-checks every null-space direction against the full non-linear constraints. On this mutant it finds a verified second witness that moves the coordinated triple $(a1x, N, y)$ and, with it, the public nullifier N . That is a real under-constraint and a real exploit: a free nullifier means one credential yields unlimited distinct pseudonyms, which defeats the sybil property the whole construction exists to provide. The adversarial harness misses it because its witness-tampering tests move one signal at a time and never perform the three-signal move the search found. So the blind-spot claim, withdrawn earlier, is reinstated, now with a concrete exploit witness rather than an inference.

The other nine mutants are not signal-level under-constraints, and the search says why. Run against the pristine deployed circuit, the two-witness search finds nine second witnesses, all at the `inv` hint inside the `IsZero` gadget of each Merkle-path comparator, free because the comparator input is zero and read by no public signal — each one proved benign by the taint analysis of §7.3, not merely assumed. It finds none that moves a public output, at any of 56 probe points. Run against the mutants: the commitment stays pinned by the Merkle path against the public root even with its assignment downgraded; the range-check mutant is an equivalent mutant, redundant against the slot-selection constraint; the four that alter the nullifier or RLN arithmetic leave the circuit deterministic and merely wrong; the two that delete a check widen the set of accepted *inputs* rather than freeing a signal, and those two the adversarial harness does catch by trying bad inputs. Restricting the mutation denominator to the one mutant the search proves is a genuine under-constraint, the honest detection rate is: of one, circomspect caught it syntactically, the adversarial harness missed it, and Picus produced no verdict. The unfiltered nine-of-ten counted signals produced by mutants that are mostly not exploitable.

The reason the mutants were hard to make exploitable is itself a result. Removing one binding constraint from this circuit frequently does not create an exploitable weakness because the signals are redundantly determined: `c` by the Merkle path, the range check by slot selection, the `IsZero` hints by nothing anyone reads. The one signal that turned out not redundantly determined is the RLN share on the residual line, which is exactly where a defect had to live: a signal with one determining constraint and no other. That over-determination also explains why several mutants are indistinguishable to behavioural testing, and it complicates any detection-rate metric over an unfiltered mutant set.

Gate 1 stays open, and mutation testing does not narrow it much. Ten mutants is a small sample chosen by the same person who wrote the circuits, which is precisely the bias an external audit exists to remove; no

mutant probed the circomlib primitives or the verifier; and a nine-of-ten detection rate on injected bugs is not a statement about the probability that the real circuits are sound. What it does provide is a better brief for an auditor than "look at everything": the classes that produce a signal, the two that only one method notices, and the two where the exercise could not establish that a bug existed at all.

7.5 A simulated memory ceiling, and where the prover actually fails

Gate 3 asks for peak resident memory and p95 on a device with 3 GB or less. The measured device has 3,751 MB, so the gate stays open. A memory ceiling can be imposed without owning such a device: run the prover inside a Linux cgroup with a hard `memory.max` and swap disabled, and lower the cap until it fails. Swap is pinned to zero deliberately, because a cgroup with swap available pages out instead of failing, which would turn a memory test into a throughput test and report a pass that a phone would not reproduce.

The result is a measured failure threshold rather than an estimate. Witness calculation plus a Groth16 prove plus verification of the deployed login circuit completes and verifies at every ceiling from 4,096 MB down to **336 MB**, charging a peak of 326 to 387 MB to the cgroup. At **320 MB** the kernel OOM-kills it. The threshold therefore sits between 320 and 336 MB, roughly an order of magnitude below the 3 GB the gate names, and the cgroup-charged peak agrees with the 361 MB peak resident memory measured on the A12. The prover also adapts downward: as the ceiling tightens from 4,096 MB to 336 MB the charged peak falls from 380 MB to 326 MB, which is the allocator and the garbage collector responding to pressure rather than a fixed working set.

This does not close Gate 3, and the gate's wording is why. It names a device, not a memory ceiling, and the distinction is load-bearing: a cgroup cap on a desktop x86 core does not reproduce a weak in-order mobile core, Android's allocator, its low-memory killer, or the smaller effective budget a browser tab receives under real pressure. Every number here is labelled simulated on desktop x86, pending confirmation on physical hardware of 3 GB or less. What the result does establish is direction: the margin is wide enough that acquiring a device is a confirmation step rather than a risk, which is the opposite of what an early warning would have looked like.

The path one step closer than a cgroup — an Android emulator capped at 2 GB and 3 GB, which runs the real Android OS and so reproduces the allocator and the low-memory killer the cgroup cannot — was attempted and not completed: the build environment could not reach the Android SDK download endpoint (repeated stalls mid-transfer). The adb-based harness (`scripts/measure_phone_rss.mjs`) is emulator-agnostic already, and the AVD setup is scripted (`scripts/wsl_setup_avd.sh`), so this is a provisioning gap on one machine, not a design gap. It would still not close the gate — an emulator reproduces the memory manager but not mobile memory bandwidth, thermals, or OEM policy — but it would move the evidence from “simulated on the wrong OS” to “measured on the right OS, pending physical hardware.”

7.6 A ceremony coordinator, built and tested, deliberately not run

Gate 5 stays open and the shipped keys stay evaluation-only. What changed is that the infrastructure to replace them exists and has been tested against deliberately invalid inputs.

A Groth16 Phase-2 ceremony is sequential. Contribution $i+1$ transforms the state that contribution i produced, and the security property, that the toxic waste is unrecoverable if any single participant destroyed their entropy, holds only if the chain really is a chain. The coordinator enforces that with four checks before

it will advance the head: the upload parses as a `zkey`; `zkey.verifyFromInit` accepts the whole chain from the frozen initial key; the submission carries exactly one more contribution than the head and the head's contribution records are a byte-exact prefix of it; and the submission is not a byte-identical replay of the head.

The third check is the one that matters and the one a naive coordinator omits. A participant can contribute onto the *initial* key rather than the current head, producing a chain of length one that passes `zkey.verifyFromInit` on its own. The test suite asserts exactly that, confirming the library call returns valid for the fork and that the coordinator rejects it anyway. Without the prefix check the ceremony is a tree, and an attacker discards every honest contribution by submitting their own short branch.

The transcript is append-only and hash-chained: each entry records the digest of the entire log preceding it, so editing an earlier entry is detectable, and the test suite doctors a rejection into an acceptance to confirm the chain breaks. Rejected submissions are recorded alongside accepted ones, because a log that records only successes is not an audit trail. A beacon step mixes in a public value that nobody could predict when the ceremony opened, which removes the last-participant advantage even if every human contributor colluded; finalization refuses to run without one.

The coordinator verifies cryptographic transcripts, so the coordinator itself can be wrong. It was tested the same way the circuits were: 27 assertions against a throwaway circuit with four honest participants and six adversarial submissions covering replay, forking from the initial key, rollback, byte corruption, two contributions bundled as one, and non-`zkey` garbage. All six are rejected, the head does not move, and an honest contribution still succeeds afterwards. **Two real bugs in the coordinator were found this way and fixed:** `snarkjs` returns false rather than raising when a beacon hash is odd-length hex or the iteration exponent falls outside its accepted range, so an unchecked call silently produces no file, and a coordinator that ignored the return value would have published a ceremony whose final beacon never happened.

It has not been run. A ceremony binds keys to one exact circuit, and that circuit changed twice in earlier work — the epoch split, then revocation. It has now been stable across four rounds of analysis, so the circuit set is frozen for v1.2: `CIRCUIT-FREEZE.md` records the source, R1CS, and constraint-count hashes, cross-checked against the reproducible build lock, and the operational sequence is written up (`docs/CEREMONY-RUNBOOK.md`). The remaining blocker is not the freeze; it is that a ceremony is only worth running with contributors who are genuinely independent of each other and of this project — the whole content of the 1-of-N-honest guarantee — and recruiting them is out of scope for the authors alone. Gate 5 moves from open with no infrastructure to open with the infrastructure built, tested, and frozen against a target, which is a smaller claim than closure and the only one the evidence supports. The shipped keys remain evaluation-only.

8. Falsification criteria

Stated in advance so that they cannot be adjusted afterwards. This work should be considered refuted if any of the following holds.

1. Credentials trade on an open market at a stable price for more than one quarter. Economic non-transferability would have failed.

2. Fewer than 3 to 5 genuinely independent attestors exist in a target jurisdiction. The diversity predicate D would be theatre, and the security argument that rests on it would be void.
3. The measured β for a target application exceeds the γ that honest users tolerate. Securing the application would price out the humans it is meant to admit.
4. A circuit audit finds an under-constrained signal permitting forged proofs.
5. No platform adopts even a zero-risk shadow-mode evaluation within 12 months. The value proposition would be wrong regardless of whether the cryptography is correct.

9. Status

What exists: a measured, working system that proves personhood privately, with reproducible builds, drop-in OIDC and W3C Verifiable Presentation integration, query-free witness maintenance, a closed prover-optimisation search, and a documented deployment matrix. Proofs from three independent prover implementations verify against one verification key with identical public signals.

What does not exist: an independent audit, a production deployment, a design partner, a measured real-world β , and closure on the five gates in Section 7 that remain open. The sixth, attester-level revocation, is closed in this version.

The accurate description is a complete, measured, unreviewed engineering system with a candidate security property. It is not a validated standard. Validation is conferred by adversaries who fail to break a system and by adopters who build on it, and neither has happened yet. Every gate in Section 7.1 is ordinary work that follows a first deployment. No cryptography remains on the critical path.

9.1 Delegation to AI agents (*not built, not analysed*)

An attested human could issue a delegated credential to a software agent, so the agent acts with attributable human backing without claiming to be a person. The primitives already exist: nullifiers in a separate namespace for agent actions, the RLN share to bound rate, and epoch scoping to bound lifetime. None of it has been built, and none of it has been analysed.

Two requirements decide whether it is sound. First, delegation must be visible. A verifier must always be able to distinguish an agent acting under delegation from a human acting directly. If it cannot, the construction becomes the bot-laundering machine it exists to prevent. Second, a per-human cap on live delegations is required. Without one, delegation multiplies the actions a single credential can take, which makes it a Sybil amplifier and defeats the economics of Section 4.

Both are stated here as requirements, not as solved problems. This direction needs its own security analysis before it is worth building.

The circuits, the reference implementation, the benchmark harnesses, the raw measurement files behind every number in Section 5, and the chronological research record including the retractions are published under Apache-2.0 with its Section 3 patent grant.

References

1. D. Chaum. Security without identification: transaction systems to make big brother obsolete. *Communications of the ACM*, 28(10):1030–1044, 1985.
2. J. Camenisch and A. Lysyanskaya. An efficient system for non-transferable anonymous credentials with optional anonymity revocation. In *EUROCRYPT 2001*, LNCS 2045, pages 93–118.
3. J. R. Douceur. The Sybil attack. In *Proceedings of the 1st International Workshop on Peer-to-Peer Systems (IPTPS)*, LNCS 2429, pages 251–260, 2002.
4. J. Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016*, LNCS 9666, pages 305–326.
5. J. H. Saltzer, D. P. Reed, and D. D. Clark. End-to-end arguments in system design. *ACM Transactions on Computer Systems*, 2(4):277–288, 1984.
6. Semaphore: a zero-knowledge protocol for anonymous signalling and double-signalling prevention on Ethereum. Privacy and Scaling Explorations, Ethereum Foundation.
7. A. Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
8. Rate-Limiting Nullifier: a construction for anonymous rate limiting using Shamir secret sharing. Privacy and Scaling Explorations, Ethereum Foundation.
9. L. Grassi, D. Khovratovich, C. Rechberger, A. Roy, and M. Schofnegger. Poseidon: a new hash function for zero-knowledge proof systems. In *USENIX Security 2021*, pages 519–535.
10. P. S. L. M. Barreto and M. Naehrig. Pairing-friendly elliptic curves of prime order. In *Selected Areas in Cryptography (SAC) 2005*, LNCS 3897, pages 319–331.
11. R. C. Merkle. A digital signature based on a conventional encryption function. In *CRYPTO '87*, LNCS 293, pages 369–378.
12. E. Ben-Sasson, A. Chiesa, E. Tromer, and M. Virza. Succinct non-interactive zero knowledge for a von Neumann architecture. In *USENIX Security 2014*, pages 781–796.

Appendix A. A real proof

Claims about zero-knowledge systems are cheap. This appendix contains an actual proof, produced by running the published artefacts, and the transcript of verifying it. A reader can regenerate all of it.

A.1 The artefacts

File	Bytes	SHA-256
build/wedge_mem_w8_d9_split.r1cs	3,756,792	fbd04ecd...83eddc27
web/wedge_mem_w8_d9_split.wasm	3,212,057	f016efcf...b4def3a1
web/wedge_mem_w8_d9_split_final.zkey	5,764,987	bd7169e4...a16acc47
web/wedge_mem_w8_d9_split_vkey.json	4,023	a4428d64...faecb356

Table 9. The R1CS and WASM hashes match `supply-chain/artifacts.lock.json`, which `scripts/repro_build.mjs --double` reproduces byte-for-byte from a clean compile of `circuits/`. The proving key is not reproducible, because it carries ceremony randomness; it is verified against the R1CS instead.

A.2 The proof

Groth16 over BN254. Three group elements, 8 field coordinates, constant size regardless of the 4,309 constraints behind it. Re-running `mkproof.mjs` produces different values here and identical values in Table 10, because Groth16 proofs are randomised. Section A.5 uses that property.

```

pi_a = [
  20845707599544265335649027855620964687374222523455596290683747059733598569414,
  1567567686339441980740655973037063527616023530367543601680152750759479006181, 1 ]
pi_b = [ [20617072746101944515302261723900367908536597911312429818557596406791920386776,
  14292379644756314352710954835857644588754862533916600688054794785036211194318],
  [14391897212804863468710221759945629259770916899971487247562052198551236621147,
  5234233511664559673585493239283667660077587361521720131010362193139911735457], [1,0] ]
pi_c = [
  19831573536458535304144255033227867520349154756296659724042617945786514259466,
  11787348736065937792382751628962761115476684537051248968393278075380426303238, 1 ]
protocol = "groth16", curve = "bn128"

```

A.3 The seven public signals

#	Signal	Value
0	N, nullifier (output)	14824683023501643632115653304856446903925650282274056340992344805449552618966
1	y, RLN share (output)	6245802399600452304690568138657552093742378804302159597634167049696237901356
2	root, tree root for epoch_tree (input)	20550735135605091646947741900747264636731978524347924805314550072189850280242
3	ctx, context (input)	987654321
4	epoch_action (input)	11
5	epoch_tree (input)	7
6	signalHash (input)	143108474036684829574744961435355602015188502350286015805048154501691818351

Table 10. That is the whole disclosure. There is no eighth signal. The commitment C, the secret s, the Merkle path, and every attester identity are private witnesses with no output wire. The verification key reports `nPublic = 7`. Signal 4 is `epoch_action`, signal 5 is `epoch_tree`, and the nullifier at signal 0 is `Poseidon(s, ctx, epoch_action)` with no dependence on `epoch_tree`.

A.4 Verification transcript

```

VERIFY: true    TAMPERED: false    nPublic: 7    proveMs: 849    verifyMs: 15

```

VERIFY: true is `snarkjs.groth16.verify(vkey, publicSignals, proof)` on the proof above. TAMPERED: false is the negative control: incrementing public signal 0 by one and re-verifying the same proof. It is rejected. A verifier that accepts both is broken, so the negative control matters as much as the positive one. Timings are Node on an i7-9850H and are not the handset figures from Section 5.

A.5 Cross-engine equivalence

The split circuit was proved by two engines on desktop: `snarkjs`, and `rapidsnark` compiled to WebAssembly (the same C++ prover from Section 5.4, run in Node). Comparing the two outputs (`scripts/xengine_split_check.mjs`):

Comparison	Result	Expected
public signals, all 7, byte-for-byte	identical	yes; the statement is the same
pi_a[0], snarkjs	218834494306333257751695454936...	—
pi_a[0], rapidsnark→WASM	967877403452917643565710914541...	differs; Groth16 proofs are randomised
both verify against the same 7-signal vkey	true	yes

Table 11. Identical public signals with different group elements is the correct outcome. Groth16 proofs are randomised, so two proofs of the same statement differ. Equivalence is established by both verifying under one verification key against the same public signals, not by the proofs matching. The `pi_a[0]` values shown are from one run and change on every run. rapidsnark native (arm64) was validated this way on the pre-split circuit on the A12 and is not re-run here; the equivalence argument is engine-independent.

A.6 Regenerating this appendix

```
npm install
node scripts/build.mjs // compiles wedge_mem_w8_d9_split (Step 2); constraint counts in Table 2
node scripts/test_rln_burn.mjs // the burn invariant: 15 assertions, all green
node scripts/xengine_split_check.mjs // cross-engine equivalence, Table 11
node integration/run.mjs // OIDC + VP drop-in, 10 assertions
node paper/mkproof.mjs // the proof and transcript in A.1 to A.4
```

Reproducing the measurements. Every figure in Section 5 traces to a JSON capture in `results/` in the accompanying repository. `scripts/repro_build.mjs --double` proves the deployed prover is byte-identical to a clean compile of the published circuits. `scripts/bench_rapidsnark.mjs`, `scripts/bench_wasmprover.mjs`, `scripts/bench_attrib_native.mjs` and `scripts/bench_attrib_wasm.mjs` reproduce Tables 4 and 5 over adb on an Android device. `scripts/measure_phone_rss.mjs` reproduces Table 3. `docs/EVALUATION.md` is a walkthrough for a reviewer verifying the core claims independently.